Universidad Nacional Autónoma de México
Facultad de Ciencias



Inteligencia Artificial

Laboratorio 2.3

Docente: *Dra. JessicaSarahiMéndezRincón*

Alumna: *OsorioLopezClaudiaYarinet*

Fecha de realización: 25/02/2026

Parte 2: Convolución

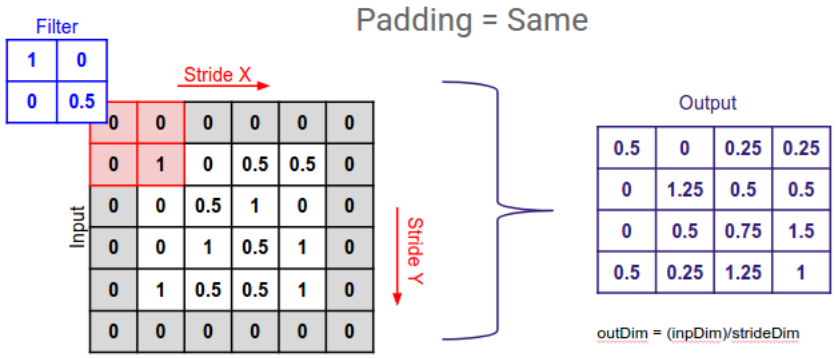
Dada la imagen I = [[1,2,0],[0,1,3],[2,1,0]]

Y el kernel K = [[1,0],[-1,1]]

Aplicar convolución 2D sin padding y stride 1.

Preguntas:

- ¿Qué cambia si se agrega padding?
- ¿Cómo impacta el tamaño del kernel en el costo computacional?




```
import numpy as np

# Definimos la imagen I y el kernel K
I = np.array([[1, 2, 0],
              [0, 1, 3],
              [2, 1, 0]])

K = np.array([[1, 0],
              [-1, 1]])

def convolution2d(image, kernel):
    img_h, img_w = image.shape
    ker_h, ker_w = kernel.shape
```

```
    # Sin padding y stride 1, el tamaño de salida es (N - K + 1)
    out_h = img_h - ker_h + 1
    out_w = img_w - ker_w + 1

    output = np.zeros((out_h, out_w))

    for i in range(out_h):
        for j in range(out_w):
            # Extraemos la región de interés (ventana)
            region = image[i:i+ker_h, j:j+ker_w]
            # Multiplicación elemento a elemento y suma
            output[i, j] = np.sum(region * kernel)

    return output

resultado = convolution2d(I, K)
print("Resultado de la convolución:")
print(resultado)
```

```
Resultado de la convolución:
[[ 2.  4.]
 [-1.  0.]]
```

¿Qué cambia si se agrega padding?

El padding (relleno de ceros en los bordes) tiene dos efectos principales:

Mantiene el tamaño: Sin padding, la imagen se encoge en cada capa de convolución. Con el padding adecuado, la salida tiene las mismas dimensiones que la entrada (3×3 en tu caso).

Información de los bordes: Sin padding, los píxeles de las esquinas solo se procesan una vez, mientras que los centrales participan en múltiples ventanas. El padding permite que la red "aproveche" mejor la información de los bordes.

¿Cómo impacta el tamaño del kernel en el costo computacional?

El impacto es cuadrático.Si tienes un kernel de $k \times k$, por cada píxel de salida realizas k^2 multiplicaciones y sumas.

Si duplicas el tamaño del kernel (de 3×3 a 6×6), el costo computacional no se duplica, sino que se cuadruplica (pasas de 9 a 36 operaciones por paso).

En modelos grandes, esto puede ralentizar significativamente el entrenamiento, por lo que a menudo se prefieren varios kernels pequeños (como 3×3) apilados en lugar de uno muy grande.

Cálculo Manual (para verificar)

Si quieres comprobar el código, el resultado debería ser:

Posición (0,0): $(1 \times 1) + (2 \times 0) + (0 \times -1) + (1 \times 1) = 2$

Posición (0,1): $(2 \times 1) + (0 \times 0) + (1 \times -1) + (3 \times 1) = 4$

Posición (1,0): $(0 \times 1) + (1 \times 0) + (2 \times -1) + (1 \times 1) = -1$

Posición (1,1): $(1 \times 1) + (3 \times 0) + (1 \times -1) + (0 \times 1) = 0$

Resultado Final:

$$\begin{bmatrix} 2 & 4 \\ -1 & 0 \end{bmatrix}$$

Ejercicio Adicional: Multiplicación de matrices de tamaño nxn

Crear una función que reciba el tamaño de la matriz n y muestre el resultado de la multiplicación. Para eso se recomienda crear matrices cuadradas de tamaño 10×10, 100×100, 1000×1000 y 10000 x10000, o hasta donde el poder de su computadora lo soporte. Llenarlas con numero aleatorios del 0 al 99 y que sirva como base para implementar una función de multiplicación de matrices. Ejecutar la multiplicación y mostrar resultados (parcialmente para tamaños grandes) del tamaño que se indique.

```
import random
import time
def crear_matriz(n: int):
    """Crea una matriz nxn con enteros aleatorios desde el 0 hasta el 99."""
    return [[random.randint(0, 99) for _ in range(n)] for _ in range(n)]

def multiplicar(A, B):
    m = len(A)
```

```
    n1 = len(A[0])
    n2 = len(B)
    p = len(B[0])

    if n1 != n2:
        raise Exception("Las matrices no se pueden multiplicar")

    # Transpuesta de B: si B es n2 x p, B_T será p x n2
    B_T = [[0 for _ in range(n2)] for _ in range(p)]

    for i in range(n2):
        filaB = B[i]
        for j in range(p):
            B_T[j][i] = filaB[j]

    result = [[0 for _ in range(p)] for _ in range(m)]

    for i in range(m):
        filaA = A[i]
        for j in range(p):
            columnaB = B_T[j]
            s = 0
            for k in range(n1):
                s += filaA[k] * columnaB[k]
            result[i][j] = s

    return result

def imprimir_matriz(M):
    for fila in M:
        print(fila)


def multiplicacion(n: int):
    if not isinstance(n, int) or n <= 0:
        raise ValueError("n debe ser un entero positivo")

    print(f"\nCreando matrices {n}x{n} con valores aleatorios 0..99...")
    A = crear_matriz(n)
    B = crear_matriz(n)

    print("Imprimiendo matriz A")
    imprimir_matriz(A)
    print("Imprimiendo matriz B")
    imprimir_matriz(B)


    print("\nMultiplicando matriz completa")
    t2 = time.time()
    C = multiplicar(A, B)
    t3 = time.time()
    print(f"Tiempo multiplicación completa: {t3 - t2:.4f} s")

    imprimir_matriz(C)




if __name__ == "__main__":
    #Caso completo: O(n³)
    try:
        n = int(input("Ingrese el tamaño n de la matriz (matriz cuadrada): "))
        multiplicacion(n)
    except ValueError as e:
        print(f"Error: {e}")
```

```
Ingrese el tamaño n de la matriz (matriz cuadrada): 5

Creando matrices 5x5 con valores aleatorios 0..99...
Imprimiendo matriz A
[59, 61, 53, 51, 79]
[86, 48, 5, 43, 96]
[53, 9, 63, 80, 10]
[78, 81, 44, 99, 22]
[17, 3, 9, 54, 31]
Imprimiendo matriz B
[71, 84, 93, 79, 71]
[81, 82, 19, 51, 14]
```

```
    [11, 9, 51, 78, 70]
    [40, 82, 95, 12, 85]
    [98, 3, 73, 74, 67]

    Multiplicando matriz completa
    Tiempo multiplicación completa: 0.0000 s
    [19495, 14854, 19961, 18364, 18381]
    [21177, 15019, 20258, 17252, 17215]
    [9365, 12347, 16643, 11260, 15769]
    [18699, 21774, 22048, 16541, 19641]
    [6747, 6276, 9490, 5140, 8546]
```

Conclusiones finales del estudiante

Cerrar

```
- ¿Qué cambia si se agrega padding?

Evita que la imagen se vea mas pequena


- ¿Cómo impacta el tamaño del kernel en el costo computacional?

En la complejidad de computacion en tiempo de ejecucion
```

- ¿Qué cambia si se agrega padding?

Evita que la imagen se vea mas pequena

- ¿Cómo impacta el tamaño del kernel en el costo computacional?

En la complejidad de computacion en tiempo de ejecucion

© 2026 UNAM Facultado de Ciencias – Todos los derechos reservados